Klinikum rechts der Isar
Technische Universität München

Multi-Modal Deep Learning

Lecture Series | Summer 2025

Institute for AI and Informatics in Medicine

06.05.2025

WHAT DOES „REPRESENTING INFORMATION“ MEAN?

Learning What a Cat Looks Like

- See specific cats (data points)

- Extract key characteristics: pointy ears, whiskers, tail, general shape (features)

- Form a reusable concept (representation) of "cat-ness"

- Forget the exact pixels of each specific cat you saw

Compress Data for General Understanding

123-123-4567

Learning a Phone Number

- There's a reusable structure: XXX-XXX-XXXX (general feature/pattern)

- The specific sequence is crucial

- Order and exact values are critical; cannot be lossily compressed

Peserve Data for Exact Recall

REPRESENTATIONS

Recap: Last week, we discussed data types and their inherent structure.

Today's Goal: Translate diverse real-world information into structured, numerical formats suitable for Deep Neural Networks (DNNs)

The Challenge:
- Real-world data comes in many forms (images, text, sound, categories...)
- DNNs primarily require numerical inputs (vectors, matrices)

Creating Representations:

- **Identify What Matters (Features)**: Select or engineer measurable characteristics relevant to the task
- **Translate to Numbers (Encoding)**: Convert these features into an initial numerical format using defined rules or techniques
- **Refine for Meaning & Efficiency (Representations/Embeddings):** Often, create richer, more compressed representations that capture deeper structure and relationships, typically learned from data

The “learning” step is basically compression of the useful, deletion of the superfluous

What makes a representation suitable for ML?

- **Informative**: Captures the essential information needed for the task, effectively discarding noise ("compressing" the signal).
- **Efficient**: Compact in size and computationally feasible to work with.
- **Meaningful**: Similar concepts/data points have similar numerical representations
- **Usable**: Directly processable by the ML model layers (e.g., numerical vectors)

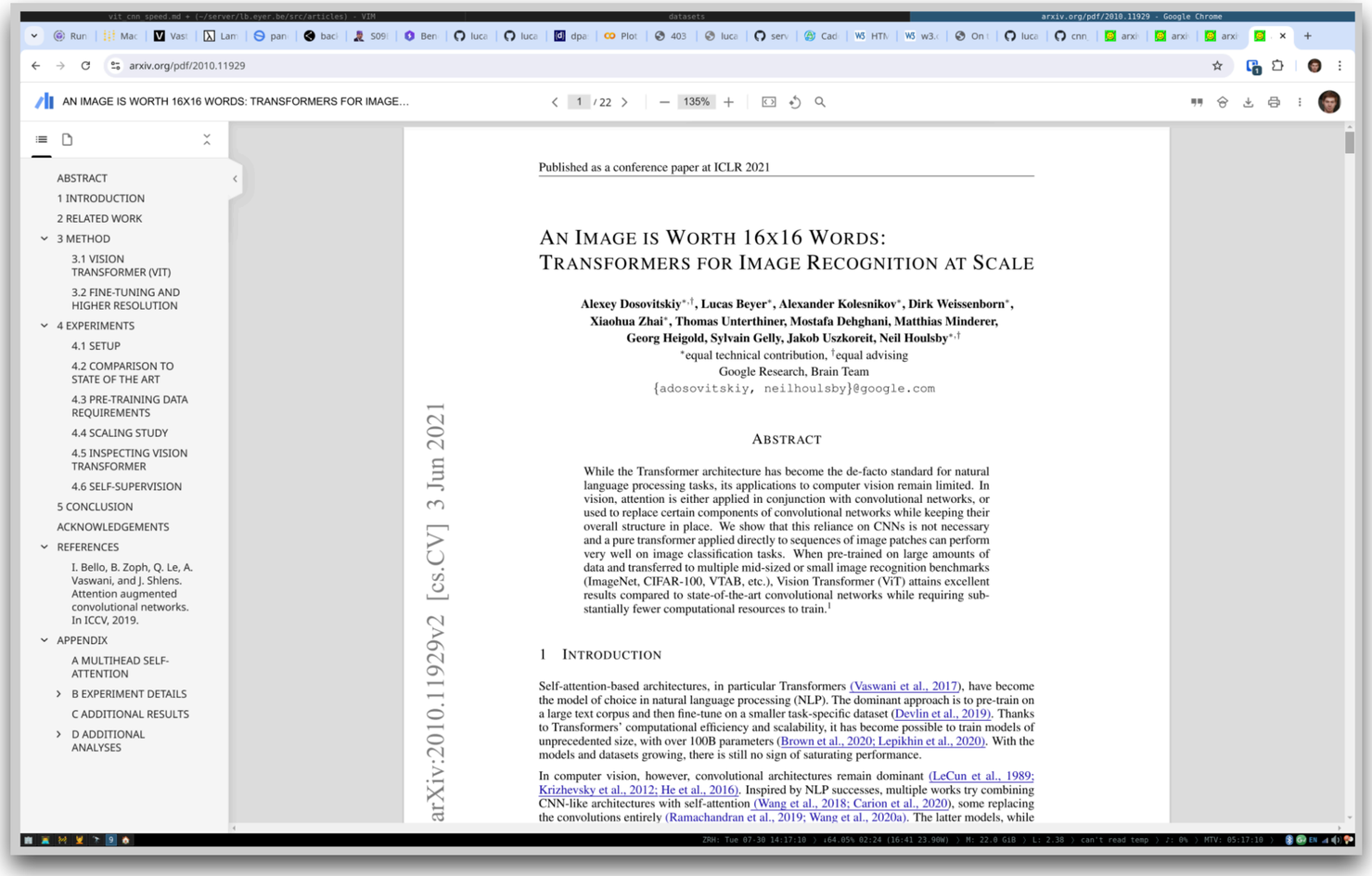
CONSTRUCTING AND RELATING REPRESENTATIONS

1
Hand-crafted/rule-based representations (rules, domain knowledge)

2
Data-driven representations (learn them)

3
Structure/regularity and relationships between representations

EXAMPLE TRANSFORMATIONS



Screenshot: 3840 x 2400 pixels with 3 color channels in float32 (*i.e. uncompressed*):

$$3840 \times 2400 \times 3 \times 4 \,\text{bytes} \approx 0.1106 \,\text{GB}$$

1000-Image Dataset: $110.6\,\text{GB}$

[2] Lucas Beyer. https://lucasb.eyer.be/articles/vit_cnn_speed.html *(accessed January 2025)*.

EXAMPLE TRANSFORMATIONS

Screenshot: 3840 x 2400 pixels with 3 color channels in float32 (*i.e. uncompressed*):

$$3840 \times 2400 \times 3 \times 4 \,\text{bytes} \approx 0.1106 \,\text{GB}$$

1000-Image Dataset: $110.6\,\text{GB}$

Representations:

A. Downsize to 224 x 224:
   ~ 0.6 GB

[2] Lucas Beyer. https://lucasb.eyer.be/articles/vit_cnn_speed.html *(accessed January 2025)*.

EXAMPLE TRANSFORMATIONS

Screenshot: 3840 x 2400 pixels with 3 color channels in float32 (*i.e. uncompressed*):

$$3840 \times 2400 \times 3 \times 4 \,\text{bytes} \approx 0.1106 \,\text{GB}$$

1000-Image Dataset: $110.6\,\text{GB}$

Representations:

A. Resize to 224 x 224:
   ~ 0.6 GB

B. JPEG image compression:
   ~ 16.0 GB

C.

[2] Lucas Beyer. https://lucasb.eyer.be/articles/vit_cnn_speed.html *(accessed January 2025)*.

EXAMPLE TRANSFORMATIONS

Screenshot: 3840 x 2400 pixels with 3 color channels in float32 (*i.e. uncompressed*):

$$3840 \times 2400 \times 3 \times 4 \text{ bytes} \approx 0.1106 \text{ GB}$$

1000-Image Dataset: 110.6 GB

Representations:

A. Resize to 224 x 224:
   ~ 0.6 GB

B. JPEG image compression:
   ~ 16.0 GB

C. Text-only (300 words):
   ~ 7.3 MB

[2] Lucas Beyer. https://lucasb.eyer.be/articles/vit_cnn_speed.html *(accessed January 2025)*.

EXAMPLE TRANSFORMATIONS

Screenshot: 3840 x 2400 pixels with 3 color channels in float32 (*i.e. uncompressed*):

$$3840 \times 2400 \times 3 \times 4 \text{ bytes} \approx 0.1106 \text{ GB}$$

1000-Image Dataset: 110.6 GB

Representations:

A. Resize to 224 x 224:
   ~ 0.6 GB

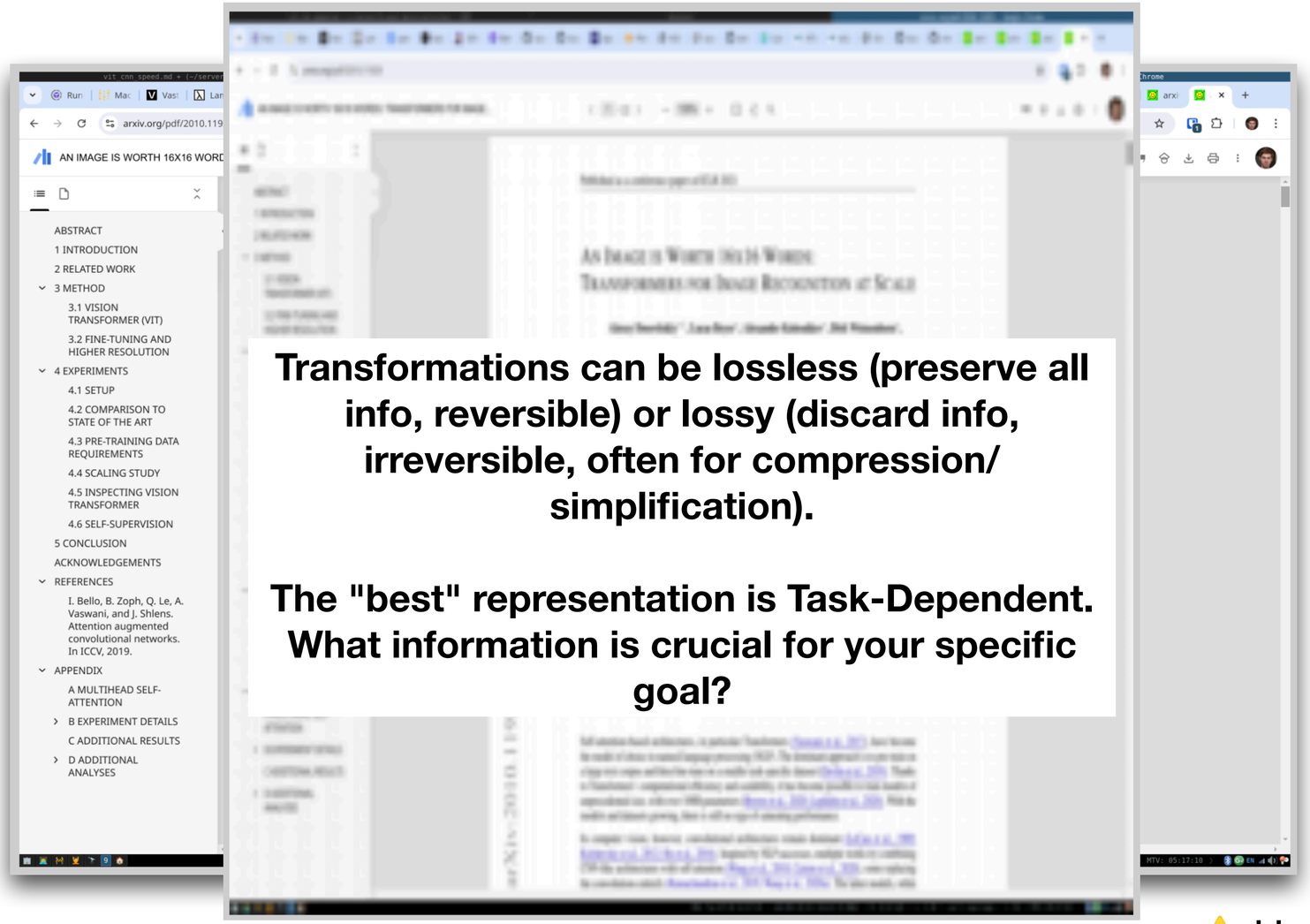
B. JPEG image compression:
   ~ 16.0 GB

C. Text-only (300 words):
   ~ 7.3 MB

⚠️ How would this work for natural images? (🐈)

[2] Lucas Beyer. https://lucasb.eyer.be/articles/vit_cnn_speed.html *(accessed January 2025)*.

EXAMPLE TRANSFORMATIONS



Transformations can be lossless (preserve all info, reversible) or lossy (discard info, irreversible, often for compression/ simplification).

The "best" representation is Task-Dependent. What information is crucial for your specific goal?

Screenshot: 3840 x 2400 pixels with 3 color channels in float32 (*i.e. uncompressed*):

$$3840 \times 2400 \times 3 \times 4 \text{ bytes} \approx 0.1106 \text{ GB}$$

1000-Image Dataset: 110.6 GB

Representations:

A. Resize to 224 x 224:
   ~ 0.6 GB

B. JPEG image compression:
   ~ 16.0 GB

C. Text-only (300 words):
   ~ 7.3 MB

⚠️ How would this work for natural images? (🐈)

[2] Lucas Beyer. https://lucasb.eyer.be/articles/vit_cnn_speed.html *(accessed January 2025)*.

ENCODINGS

1. Deep Learning Needs Numbers
    - Deep learning models are built on mathematical operations
    - Core components: Batched Matrix Multiplications (BMMs) and element-wise non-linearities (activation functions)
    - These operations fundamentally require data to be represented as numerical vectors, matrices

2. Raw Data Isn't Always Numerical

- Some data aligns easily:
    - Images: Pixel grids (Matrices)
    - Time Series: Sequences of measurements (Vectors/Matrices)
- Many data types don't:
    - Categorical Labels: Discrete classes ('cat', 'dog', 'healthy', 'diseased')
    - Text: Sequences of words/characters
    - Graphs: Nodes and edges (Relationships)
    - Point Clouds: Unordered sets of 3D points
    - Tabular Data: Often contains mixed types (text, numbers, categories)

1 Hand-crafted/rule-base representations

➡️ Convert data into suitable format for deep learning by leveraging domain knowledge to highlight important patterns or relationships hidden in the raw data.

➡️ Domain heuristics can pre-shape the data (feature engineering). Good when data volume is small or interpretability matters.

ENCODING CATEGORICAL DATA

💡 Systematic way of transforming data for efficient storage, processing, or transmission.

Label	Categorical #
Diabetic Macular Edema	1
Drusen	2
Healthy Retina	3

One-hot encoding →

Diabetic Macular Edema	Drusen	Healthy Retina
1	0	0
0	1	0
0	0	1

One-Hot Encoding:

⚠️ Sparse vector representation of categorical data of known number of categories.

- Simple yet widely used encoding to represent label information.
- Allows for categorical data to be used within linear algebra, probability theory etc. (orthogonal)
- Large one-hot matrices can be memory-inefficient if not appropriately encoded (specialized representation)

ENCODING CATEGORICAL DATA

💡Systematic way of transforming data for efficient storage, processing, or transmission.

Label	Categorical #
Diabetic Macular Edema	1
Drusen	2
Healthy Retina	3

One-hot encoding →

Diabetic Macular Edema	Drusen	Healthy Retina
1	0	0
0	1	0
0	0	1

Numerical: $x, y \in \mathbb{R}, \mathbf{w} \in \mathbb{R}^n : y = x \cdot \mathbf{w} = \sum_{i=0}^{n} x \cdot w_i$

- Assumes relative ordering between categories (*think scaling of input values*)
- Output of a linear layer scales with the **order** of the inputs

One-hot: $x \in \mathbb{R}; \mathbf{y} \in \{[0,0,1],[0,1,0],[1,0,0]\}, \mathbf{w} \in \mathbb{R}^n : \mathbf{y} = [y_0, y_1, y_2]^\top = x \odot \mathbf{w} = [x \cdot w_0, x \cdot w_1, \ldots, x \cdot w_n]^\top$

- Learns distinct features for categories
- More flexibility adding/removing categories
- No assumptions about categorical relationship: no scaling of the linear layer output

ENCODING TEXT DATA

💡Systematic way of transforming data for efficient storage, processing, or transmission.

Tokens: encode plaintext into "tokens" which are natural numbers

n-grams (sliding windows):

⚠️ Convert text into structured numerical data.

- Break up sequences of words (or characters) into fixed-length contiguous subword units
- Limited context and generalization, very high dimensional: bigger n captures more context but explodes the number of possible n-grams
- Feature extraction – for text classification, n-gram counts give a bag-of-phrases representation

Example: „multimodal“

- **1-gram (unigram)**:

['m', 'u', 'l', 't', 'i', 'm', 'o', 'd', 'a', 'l']

- **2-gram (bigram)**:

['mu', 'ul', 'lt', 'ti', 'im', 'mo', 'od', 'da', 'al']

- **3-gram (trigram)**:

['mul', 'ult', 'lti', 'tim', 'imo', 'mod', 'oda', 'dal']

ENCODING TEXT DATA

💡Systematic way of transforming data for efficient storage, processing, or transmission.
Tokens: encode plaintext into "tokens" which are natural numbers

Byte-Pair Encodings (BPE):

⚠️ Flexible subword units based on frequency.

- Iteratively merges the most frequent adjacent character pairs to build a vocabulary of subword units.
- Solves sparsity problem in NLP
- Handles out of vocabulary words
- Newer technique: Work on (e.g. unicode) bytes directly (compare tiktoken or GPT-2-BPE)

ENCODING TEXT DATA

💡Systematic way of transforming data for efficient storage, processing, or transmission.
Tokens: encode plaintext into "tokens" which are natural numbers

Byte-Pair Encodings (BPE):

```
"hug", "pug", "pun", "bun", "hugs"
```

corpus uses these five words

base vocabulary: ["b", "g", "h", "n", "p", "s", "u"]

```
("hug", 10), ("pug", 5), ("pun", 12), ("bun", 4), ("hugs", 5)
```

assume the words had the following frequencies

```
("h" "u" "g", 10), ("p" "u" "g", 5), ("p" "u" "n", 12), ("b" "u" "n", 4), ("h" "u" "g" "s", 5)
```

see each word as a list of tokens

most frequent pair ("u", "g"), total of 20 times

```
Vocabulary: ["b", "g", "h", "n", "p", "s", "u", "ug"]
Corpus: ("h" "ug", 10), ("p" "ug", 5), ("p" "u" "n", 12), ("b" "u" "n", 4), ("h" "ug" "s", 5)
```

("u", "g") -> "ug"

most frequent pair is ("u", "n")

```
Vocabulary: ["b", "g", "h", "n", "p", "s", "u", "ug", "un"]
Corpus: ("h" "ug", 10), ("p" "ug", 5), ("p" "un", 12), ("b" "un", 4), ("h" "ug" "s", 5)
```

("u", "n") -> "un"

most frequent pair is ("h", "ug")

```
Vocabulary: ["b", "g", "h", "n", "p", "s", "u", "ug", "un", "hug"]
Corpus: ("hug", 10), ("p" "ug", 5), ("p" "un", 12), ("b" "un", 4), ("hug" "s", 5)
```

("h", "ug") -> "hug"

Continue like this until reaching the desired vocabulary size.

Example from: https://huggingface.co/learn/llm-course/chapter6/5

NUMERIC DATA

Often encounter numeric data directly: stock prices, medical readings, social media counts (likes, shares), etc.

Raw numeric values often have issues:

Scale mismatch: one column ranges from 0–1 while another spans 10 000 – 1 000 000. Large-magnitude features can dominate distance or gradient calculations

Different distributions: many optimization routines converge faster when inputs are roughly centered around 0

Normalization / Standardization:

- **Normalization (often Min-Max Scaling):** Rescaling data to a fixed range, typically [0, 1] or [-1, 1]
- **Formula: X_scaled =** (X - X_min) / (X_max - X_min)
- **Standardization (Z-score Normalization):** Rescaling data to have a mean of 0 and a standard deviation of 1
- **Formula: X_scaled =** (X - mean) / std_dev

Why Normalize/Standardize?

- **Fair feature contribution –** prevents any single, large-scale feature from overpowering others
- **Algorithmic stability & speed –** centred, similar-scale inputs lead to faster, more stable convergence for GD-based methods
- **Interpretability –** after standardization, feature coefficients roughly reflect “effect per standard deviation” making cross-feature comparisons easier

ENCODING HIGHER-DIMENSIONAL DATA

💡 Encodings can also be higher-dimensional, e.g. for imaging data.

Radiomics:

Extraction of quantitative features from medical images to describe tumors or tissue.

- **Key Idea:** quantify characteristics not readily apparent to human eye by visual assessment
- **Workflow:** Image segmentation (ROI) → feature extraction → data analysis
- **Goal:** Predictive biomarkers based on pixel values

Feature Extraction:

- Use combination of transformations (*e.g. statistical, spatial, and frequency-based*) to capture patterns.
  - Example: Entropy of the pixel/voxel histogram
- Compute intensity, texture, and shape descriptors from segmented medical images.

ENCODING HIGHER-DIMENSIONAL DATA

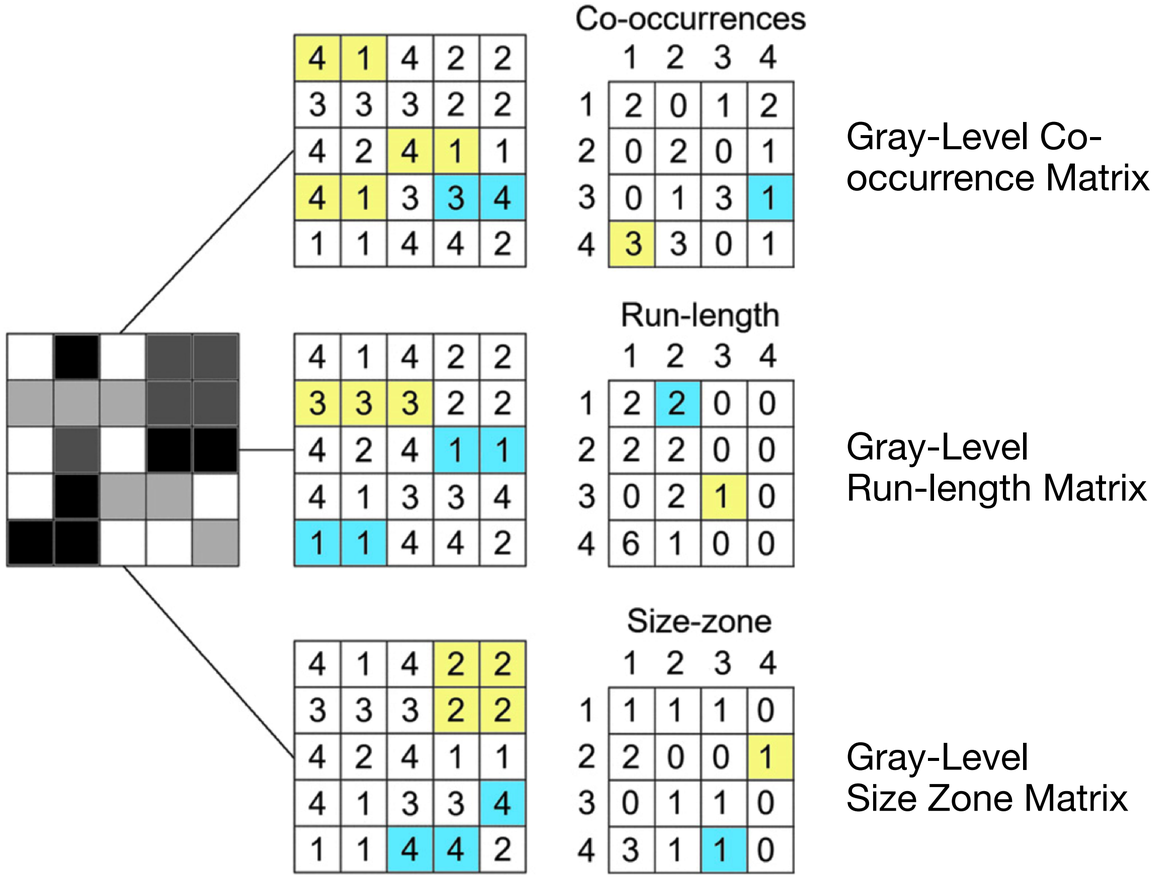
💡 Encodings can also be higher dimensional, e.g. for imaging data.

Radiomics:

Extraction of quantitative features from medical images to describe tumor/tissue.

⚠️ Radiomics features are human-defined.

- Human bias
- Not comprehensive due to limited complexity
- Lack of adaptability



[4] Mayerhoefer et al. Introduction to Radiomics *(Journal of Nuclear Medicine 2020)*.

ENCODING COMPLEX INPUTS AS LINEAR COMBINATIONS OF “BASIS VECTORS”

- The idea of feature detectors, like edge detectors in Radiomics activating in certain regions, relates to a broader concept: decomposing a complex input into a **combination of simpler 'basis vectors'**
- The Fourier transform provides a powerful mathematical framework for this, decomposing signals into a sum of sine and cosine waves (sinusoids) of different frequencies. These sinusoids act as basis functions

FOURIER FEATURE ENCODING (FFE)

💡An alternative preprocessing technique for numerical features
Goal: Enrich input space, gain normalization-like effects & numerical stability without dataset statistics
Maps low-dimensional numerical inputs to a higher-dimensional space

How FFE Works (for a feature x):

- Define a set of increasing scales (e.g., 2^{-1}, 2^0, 2^1, 2^2, 2^3, 2^4, 2^5, 2^6)
- Divide x by each scale: x/s_1, x/s_2, ... x/s_n
- Apply sin and cos to each scaled value
- Result: A single number x is transformed into a multi-dimensional vector:

$$x \mapsto \left(\sin(2x), \cos(2x), \sin(x), \cos(x), \frac{\sin(x)}{2}, \frac{\cos(x)}{2}, \ldots, \frac{\sin(x)}{2^n}, \frac{\cos(x)}{2^n} \right)$$

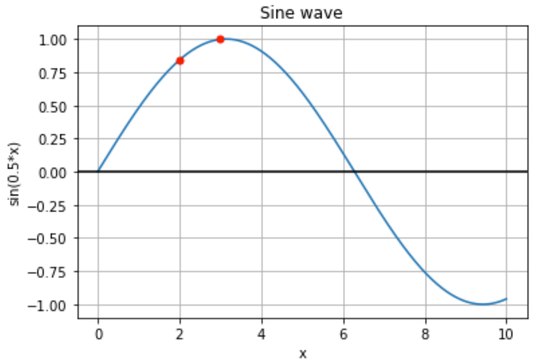
[3] https://sair.synerise.com/fourier-feature-encoding/*(accessed January 2025)*.

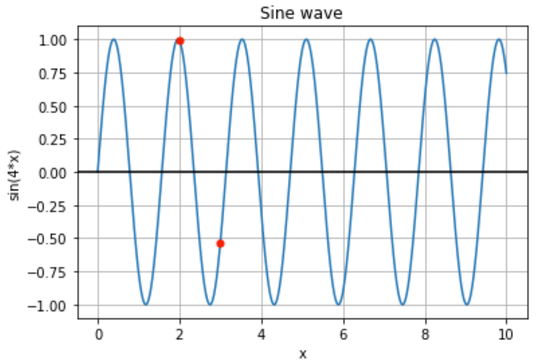
FOURIER FEATURE ENCODING (FFE)

💡An alternative preprocessing technique for numerical features.
Goal: Enrich input space, gain normalization-like effects & numerical stability without dataset statistics.
Maps low-dimensional numerical inputs to a higher-dimensional space.

Why FFE is Beneficial

- Multi-Scale Sensitivity: sin/cos functions with different scales (frequencies) capture changes in the input value at different granularities:
- We can see that the lower frequency function will react to this change in the lower degree and the higher frequency function will capture this change better. By combining functions with different frequencies we represent different levels of sensitivity for changes in input feature value.
- Richer Representation: Maps input to a higher-dimensional space with more complex geometry
- Improved Learning: Can lead to richer gradients and more efficient backpropagation
- Normalization-like Effect: Provides numerical stability similar to normalization

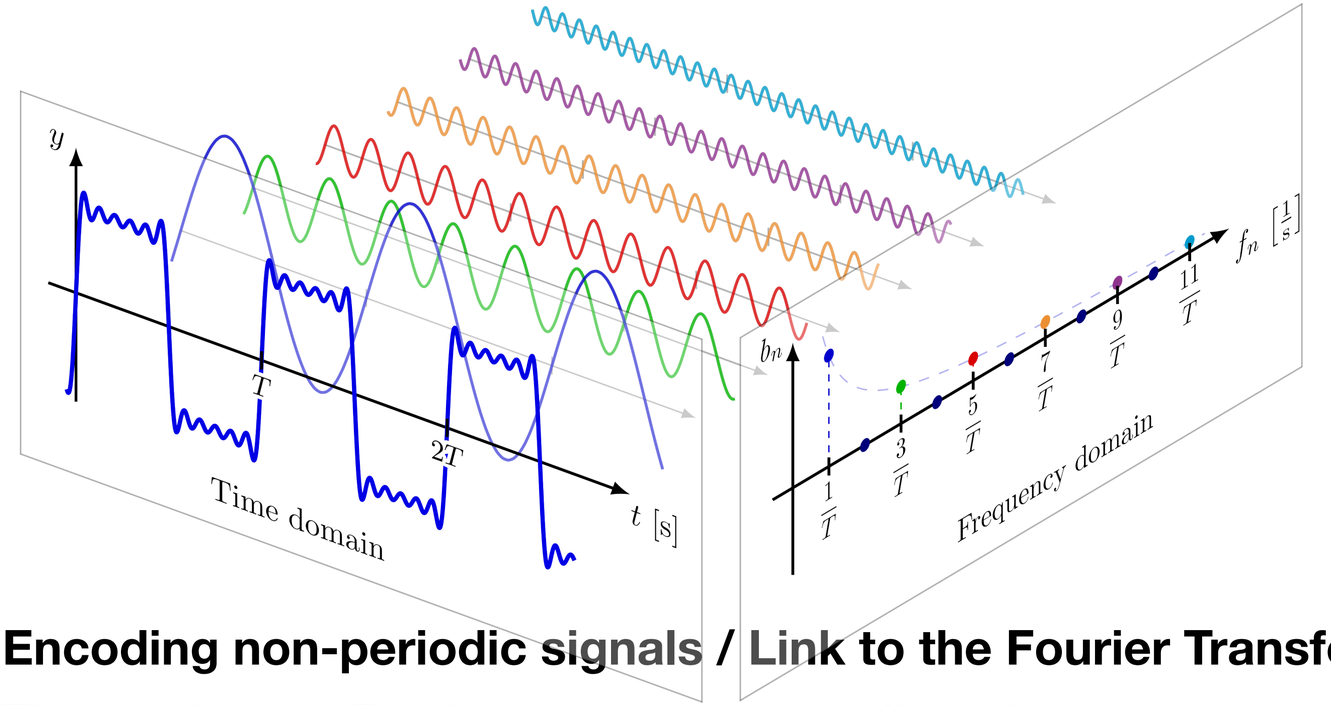




[3] https://sair.synerise.com/fourier-feature-encoding/*(accessed January 2025)*.

FOURIER FEATURE ENCODING (FFE)

Link to Fourier Analysis: any **periodic** function can be written as a sum of sine and cosine waves (Fourier series)



$$f(t) = \sum_{k=0}^{\infty} \Big(a_k \cos(k\omega_0 t) + b_k \sin(k\omega_0 t) \Big)$$

Here, $\{\cos(k\omega_0 t), \sin(k\omega_0 t)\}$ form an orthogonal basis over one period.

Fourier Feature Encoding chooses a finite set of those basis functions (e.g. k = –1…6) and plugs the scalar x in for t:

$$x \mapsto \left(\sin(2x), \cos(2x), \sin(x), \cos(x), \frac{\sin(x)}{2}, \frac{\cos(x)}{2}, \ldots, \frac{\sin(x)}{2^n}, \frac{\cos(x)}{2^n} \right)$$

Encoding non-periodic signals / Link to the Fourier Transform:

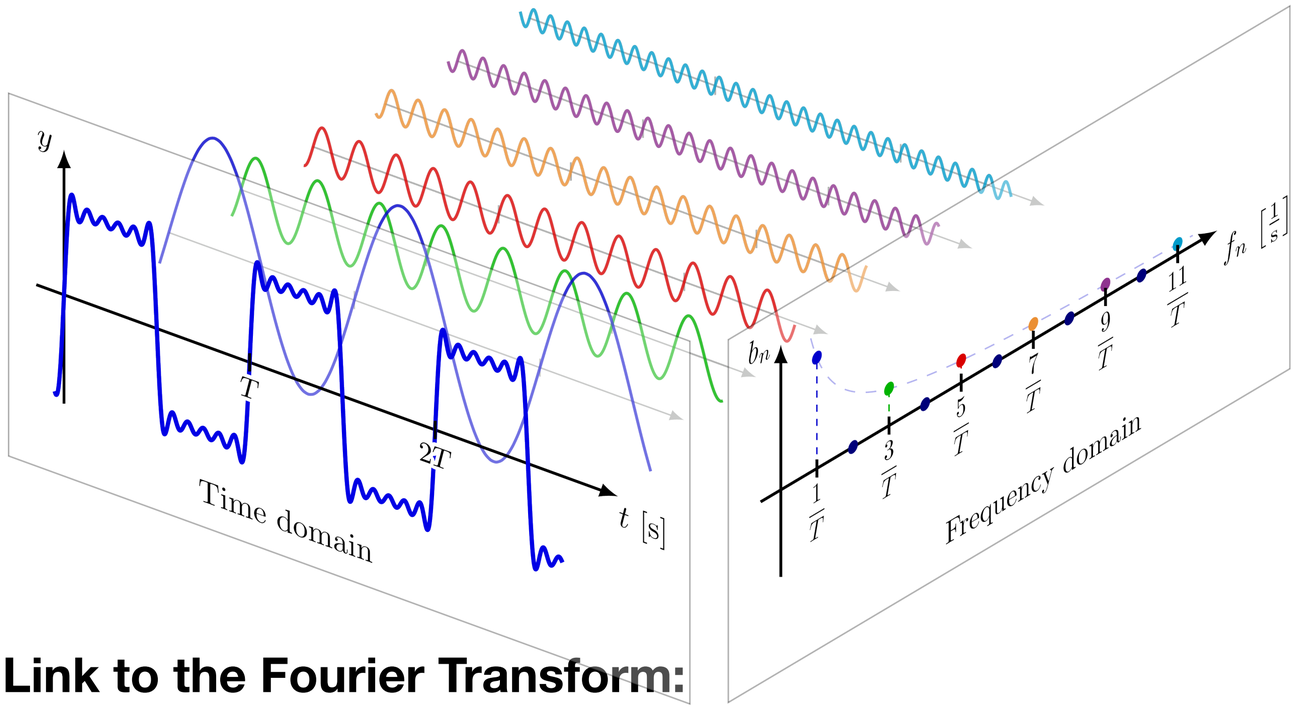
The continuous Fourier transform generalises the series to non-periodic signals:

For a continuous signal $x(t)$, the frequency components $X(f)$ are given by: $X(f) = \int_{-\infty}^{\infty} x(t) e^{-2\imath\pi f t} dt$

[3] Duke Institute for Brain Sciences Methods. https://dibsmethodsmeetings.github.io/fourier-transforms *(accessed January 2025)*.

ENCODING SIGNALS

Link to Fourier Analysis: any **periodic** function can be written as a sum of sine and cosine waves (Fourier series)



$$f(t) = \sum_{k=0}^{\infty} \Big(a_k \cos(k\omega_0 t) + b_k \sin(k\omega_0 t) \Big)$$

Here, $\{\cos(k\omega_0 t), \sin(k\omega_0 t)\}$ form an orthogonal basis over one period.

Fourier Feature Encoding chooses a finite set of those basis functions (e.g. k = –1…6) and plugs the scalar x in for t:

$$x \mapsto \left(\sin(2x), \cos(2x), \sin(x), \cos(x), \frac{\sin(x)}{2}, \frac{\cos(x)}{2}, \ldots, \frac{\sin(x)}{2^n}, \frac{\cos(x)}{2^n} \right)$$

Link to the Fourier Transform:

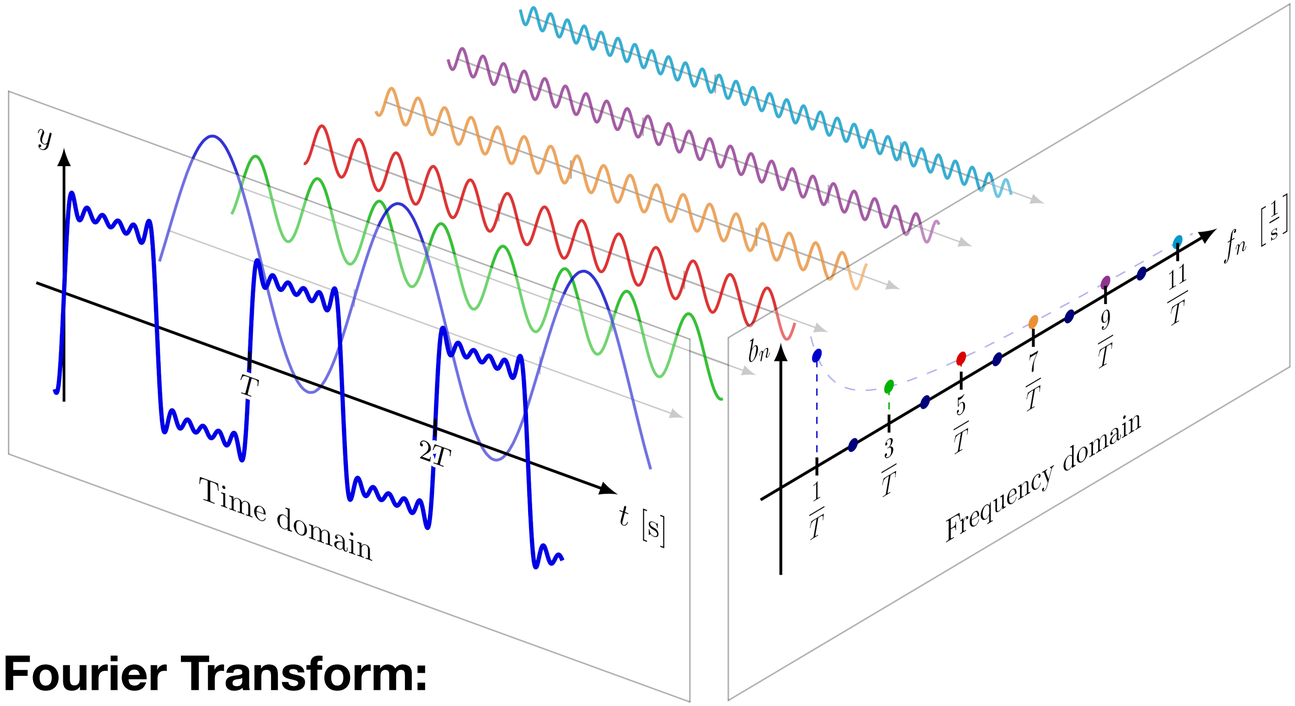
⚠ Allows for representing signals as a vector of frequency components - each element corresponding to a sinusoidal basis function in the frequency domain.

- Efficient data compression and noise reduction by isolating dominant frequency components.
- Highlights periodic patterns and converts convolution operations into multiplications.

[3] Duke Institute for Brain Sciences Methods. https://dibsmethodsmeetings.github.io/fourier-transforms *(accessed January 2025)*.

FOURIER TRANSFORM

💡Systematic way of transforming data for efficient storage, processing, or transmission.



Frequency Domains
1
0
1
1

→

Frequency Band 1	Frequency Band 2	Frequency Band 3
1	0	0
0	0	0
0	1	0
0	0	1

Fourier Transform:

⚠️ You can think of the frequency components derived from the Fourier Transform as an encoding – a vector indicating the 'presence' or 'strength' of different frequency basis functions, somewhat analogous to how one-hot encoding indicates category presence. This representation enables compression, highlights patterns, and simplifies operations

[3] Duke Institute for Brain Sciences Methods. https://dibsmethodsmeetings.github.io/fourier-transforms *(accessed January 2025)*.

- Fourier series are a specific linear decomposition that is especially suitable for periodic signals but applicable to all kinds of inputs
  - The “basis functions” are phase-shifted sinusoids
  - We “decompose” the input into a (finite - DFT) sum of those phase-shifted sinusoids
  - Sinusoids are efficient to represent, so we can represent inputs with fewer bits
  - JPEG works similarly (discrete cosine transform (DCT) expresses a finite sequence of data points in terms of a sum of cosine functions oscillating at different frequencies)
- Important intuitions:
  - The feature space of a CNN is quite similar to a Fourier space, only that the “basis vectors” (filters as basis) are learned during training rather than fixed as sinusoids
  - Convolutions are multiplications in Fourier space

REPRESENTATIONS OF DATA

2

Data-driven representations

➡️ Can we ditch the hand-crafted representations and learn useful features **directly from the data?**

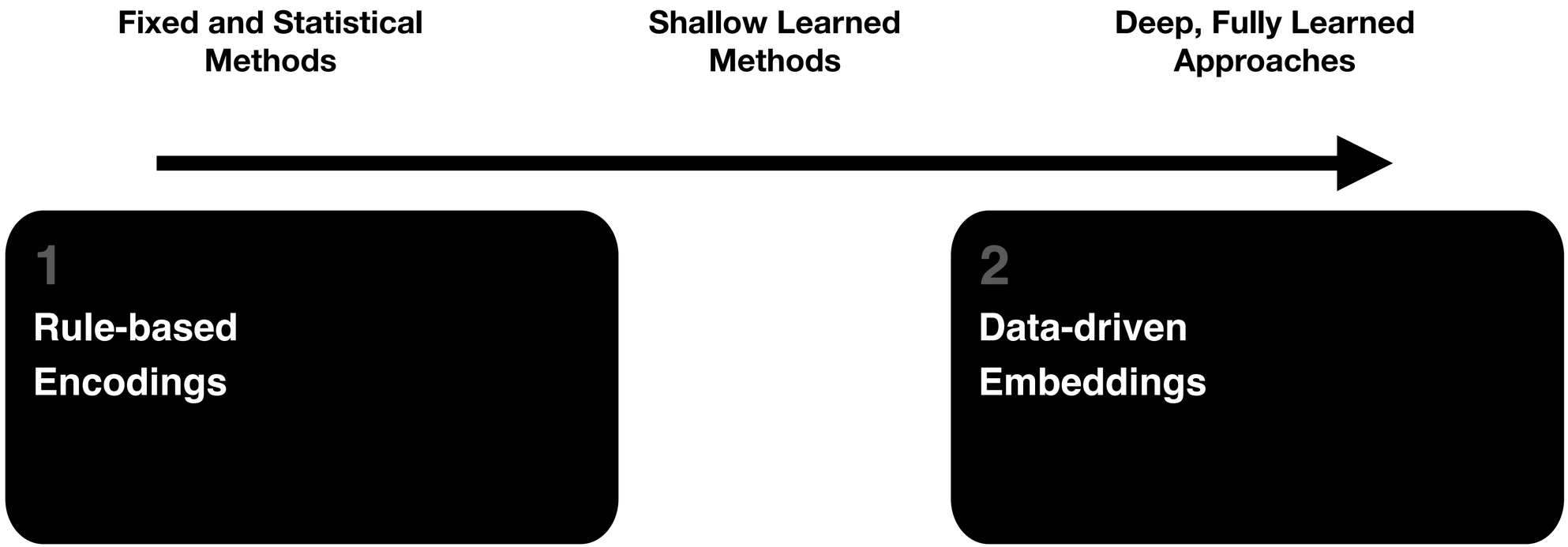
➡️ How can we further compress data while keeping or even expose meaningful information? Spoiler: usually yes.

REPRESENTATIONS OF DATA

➡️ How can we further compress data while keeping or even expose meaningful information?

⚠️ No clear distinction, but:

Helpful Ranking: from *shallow deterministic methods* to increasingly *learned dense approaches.*



REDUCING DIMENSIONS THROUGH MATRIX MULTIPLICATION

💡 **Embeddings:** Mapping from one space to another, to capture semantic or structural relationships.

⚠️ Can high-dimensional data be represented in a lower-dimensional space without losing its essential structure?

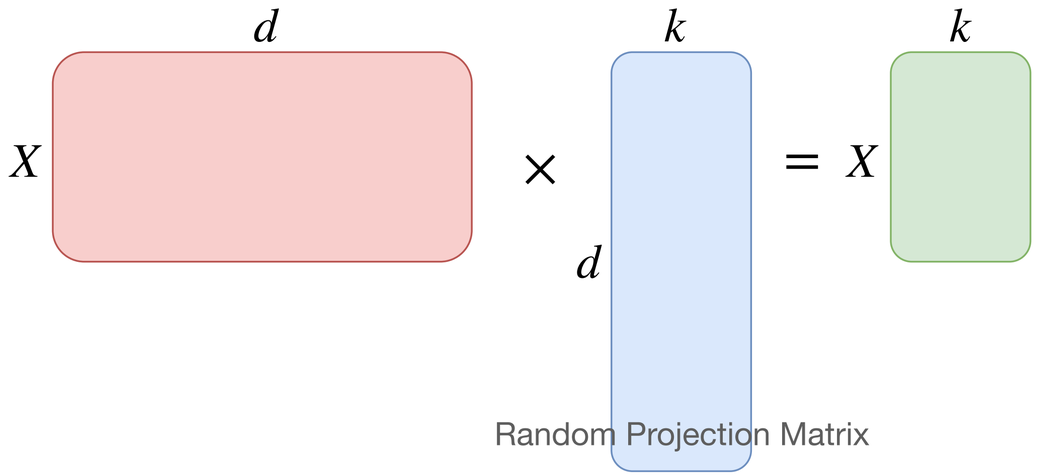
🤔 Mostly!

Johnson-Lindenstrauss Lemma:

Let X be a set of n points in $\mathbb{R}^d$. For any $0 < \varepsilon < 1$, there exists a mapping $f : \mathbb{R}^d \to \mathbb{R}^k$ such that for all $x, y \in X$:

$$(1 - \varepsilon)\|x - y\|^2 \leq \|f(x) - f(y)\|^2 \leq (1 + \varepsilon)\|x - y\|^2,$$

with $k = O\left(\frac{\log n}{\varepsilon^2}\right)$.



Random Projection Matrix

REDUCING DIMENSIONS THROUGH MATRIX MULTIPLICATION

💡 **Embeddings:** Mapping from one space to another, to capture semantic or structural relationships.

⚠️ Can high-dimensional data be represented in a lower-dimensional space without losing its essential structure?

Johnson-Lindenstrauss Lemma:

Let X be a set of n points in $\mathbb{R}^d$. For any $0 < \varepsilon < 1$, there exists a mapping $f : \mathbb{R}^d \to \mathbb{R}^k$ such that for all $x, y \in X$:

$$(1-\varepsilon)\|x-y\|^2 \leq \|f(x)-f(y)\|^2 \leq (1+\varepsilon)\|x-y\|^2,$$

with $k = O\left(\frac{\log n}{\varepsilon^2}\right)$.

Implications:

- A set of points can be embedded into a space of (much) lower dimension while preserving the pairwise distances within a bounded error (*controlled distortion).*
- Many kinds of simple randomised projections (*e.g. multiplication with any sub-Gaussian random matrix*) allow for reducing the dimensionality within this bound.

The JL Lemma has practical uses, for example, in large language models.
We can suggest this video as a nice application of the JL transform in LLMs: https://www.youtube.com/watch?v=9-Jl0dxWQs8&t

INTRINSIC DIMENSION OF DATA

💡 **Embeddings:** Mapping from one space to another, to capture semantic or structural relationships.

⚠️ Further reduce dimensionality to enhance representation and usability.

Key Idea of Manifold Hypothesis:

High-dimensional data (*e.g., images, text, sensor data*) often has low-dimensional structure, *i.e.* lies near a low-dimensional manifold within the feature space.

- **Extrinsic Dimension**: The raw feature space dimension (e.g., 1M pixels in an image $\rightarrow 10^6$ -dimensional space).
- **Intrinsic Dimension**: The minimum number of parameters needed to describe the data without significant loss of information.

Implications:

Learn compact embeddings which increase usability instead of treating all dimensions as equally important.

Random projection is one way to reduce dimensions. But can we do better? Can we find a representation that's not just lower-dimensional but also captures the inherent structure more effectively?

[4] Landauer et al. An Introduction to Latent Semantic Analysis. *(Discourse Processes 1998)*.

RANDOMIZED PROJECTIONS AND SINGULAR VALUE DECOMPOSITION ARE BOTH LINEAR MATRIX OPERATIONS.

While random projections preserve pairwise distances, they don't necessarily find the most meaningful or interpretable lower-dimensional representation. The resulting dimensions in the k-dimensional space might not correspond directly to intuitive concepts. The projection is random, not optimized for any specific task or interpretation beyond preserving the overall geometry.

💡 **Recap:** SVD decomposes a matrix $X \in \mathbb{R}^{m \times n}$ into three matrices: $X = U\Sigma V^T$

singular values (*importance*) singular vectors (*structure*)

Intrinsic Dimension and Structure:

Find an optimal basis (*coordinate system*) that captures meaningful structure in data.

$$X \approx AB$$

With A being the basis of the system (*embedding matrix*) and B the projected data in the system (*coefficients*).

RANDOMIZED PROJECTIONS AND SINGULAR VALUE DECOMPOSITION ARE BOTH LINEAR MATRIX OPERATIONS.

💡 **Recap:** SVD decomposes a matrix $X \in \mathbb{R}^{m \times n}$ into three matrices: $X = U\Sigma V^T$

singular values (*importance*) singular vectors (*structure*)

Intrinsic Dimension and Structure:

Find an optimal basis (*coordinate system*) that captures meaningful structure in data.

$$X \approx AB$$

With A being the basis of the system (*embedding matrix*) and B the projected data in the system (*coefficients*).

UNCOVERING STRUCTURE THROUGH MATRIX DECOMPOSITIONS

💡 **Recap:** SVD decomposes a matrix $X \in \mathbb{R}^{m \times n}$ into three matrices: $X = U\Sigma V^T$

singular values (*importance*) singular vectors (*structure*)

Intrinsic Dimension and Structure:

Find an optimal basis (*coordinate system*) that captures meaningful structure in data.

$$X \approx AB$$

With A being the basis of the system (*embedding matrix*) and B the projected data in the system (*coefficients*).

Example:

SVD captures variance within data - which can be use to reduce the rank k of any matrix $X, X' \in \mathbb{R}^{m \times n}$:

$$X_k{}' = AB'B = U_k\Sigma_k(\Sigma_k^{-1}U_k^T X' V_k)V_k^T = U_k(U_k^T X' V_k)V_k^T \,,$$

with $A = U_k\Sigma_k$ and $B = V_k^T$.

⚠️ Capture the most informative components while reducing noise and redundancy.

PRINCIPAL COMPONENT ANALYSIS (PCA)

Goal: Reduce dimensionality of data while preserving maximum variance. Find the principal axes (directions of greatest variance) in the data.

PCA can be efficiently and numerically stably computed using SVD.

If X is the mean-centered data matrix (Center X by subtracting its column-mean):

Perform SVD on X: $X = U\Sigma V^T$

UNCOVERING STRUCTURE THROUGH MATRIX DECOMPOSITIONS

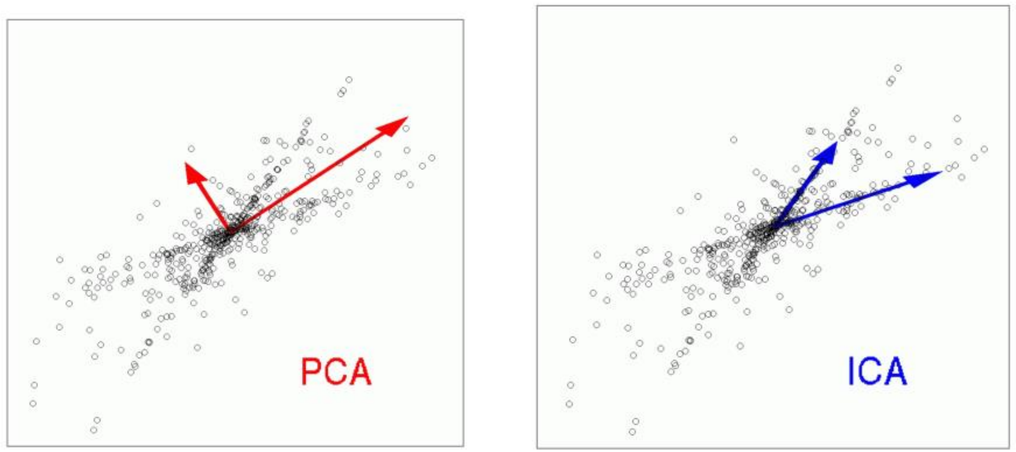
💡 **Recap:** SVD decomposes a matrix $X \in \mathbb{R}^{m \times n}$ into three matrices: $X = U\Sigma V^T$

singular values (*importance*) singular vectors (*structure*)

⚠️ Capture the most informative components while reducing noise and redundancy.

Similar concept in other approaches:

- Independent Component Analysis (*maximizes statistical independence by minimizing mutual information*)
- Spectral Embeddings (*e.g. t-SNE, UMAP - optimize for local and global geometric structure*)
- Neural Network Latents (*non-linear - reference coordinate system is learned*)



[5] Barnabás Póczos. Introduction to Independent Component Analysis and Its Generalizations. *(University of Alberta 2009)*.

MORE ON NON-LINEARITY

Random projections show we can probabilistically keep distances in fewer dimensions.

SVD/PCA show we can deterministically keep the directions that matter most.

How can we create more expressive mappings?

Randomized Projections, SVD, PCA are entirely linear processes.

➡️ Move beyond pure linear algebra: add non-linearity and optimisation to learn embeddings that capture semantics impossible to recover with fixed linear map

1️⃣ Adapt *how we learn* the embeddings.

2️⃣ Make embeddings themselves non-linear.

Let's focus on 1️⃣ first…

Note:

Embeddings - projections into a different-dimensional vector space

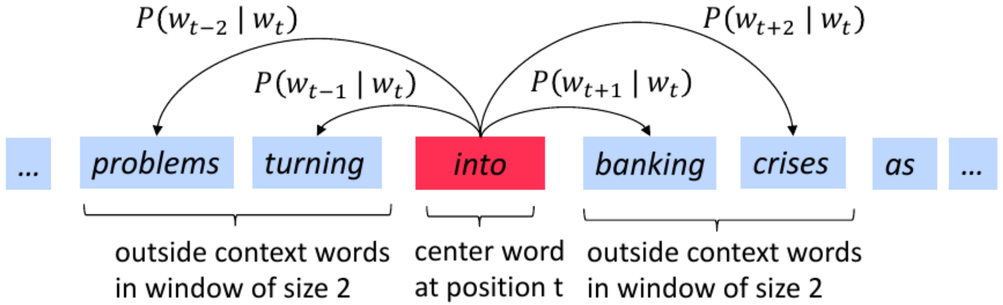
LEARNING EMBEDDINGS THROUGH OPTIMIZATION

💡 To capture more complex semantic relationships, learning the embedding is required.

Word2Vec

Learn word embeddings by predicting a target word w_t given it's context words $C_t = \{w_{t-c}, \ldots, w_{t+c}\}$ (*CBoW*).

Optimize: $J(\theta) = \sum_{t=1}^{T} \log P(w_t \mid C_t)$



$$P(w_t \mid C_t) = \frac{\exp\left(v_{w_t}^T \cdot \frac{1}{2c} \sum_{w \in C_t} U x_w\right)}{\sum_{w \in V} \exp\left(v_w^T \cdot \frac{1}{2c} \sum_{w \in C_t} U x_w\right)}$$

, with x_w the one-hot encoded word, embedding matrix U, and v_w a word of the vocabulary V in the projected output space.

⚠️ While the Word2Vec embedding is linear, the optimization itself is non-linear.

Foreshadowing: the first layer of LLMs today behave similarly - but with different learning signal.

[6] Word2Vec paper: https://arxiv.org/abs/1301.3781.

INTRODUCING NON-LINEARITY

How can we create *even* more capable mappings?

➡️ Introduce non-linearity to create more complex mappings.

1️⃣ Adapt *how we learn* the embeddings.

2️⃣ Make embeddings themselves non-linear.

Combine 1️⃣+2️⃣ …

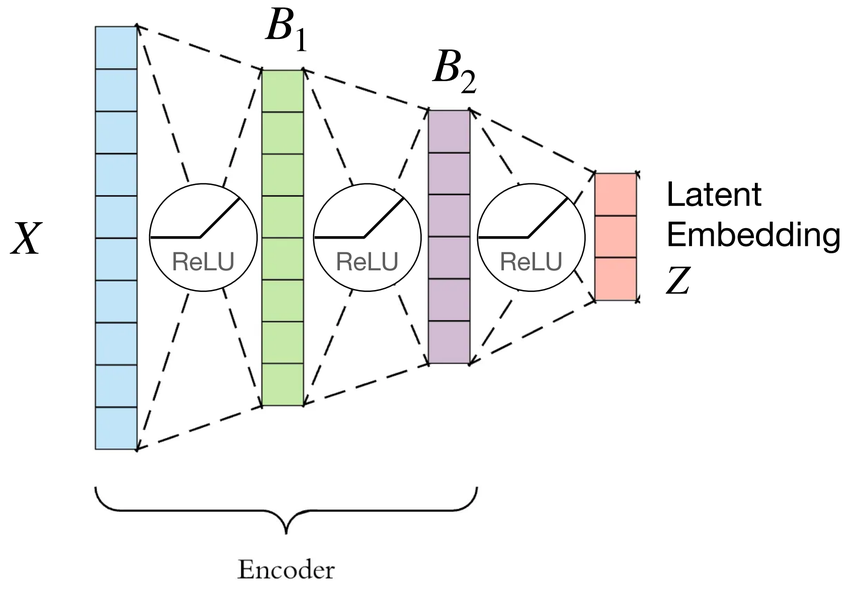
LEARNING NON-LINEAR EMBEDDINGS

💡 To capture **even more** complex semantic relationships, learning **non-linear** embeddings is required.

Neural Network Encoders

Modern encoders learn embeddings through hierarchical, structured, and compositional non-linear transformations.

- **Note:** single-layer MLP without activation is simply a linear mapping ($B = AX + \beta$).
- Each intermediate representation of an encoder is an embedding.



[7] Arden Dertat. https://medium.com/towards-data-science/applied-deep-learning-part-3-autoencoders-1c083af4d798 *(accessed January 2025)*.

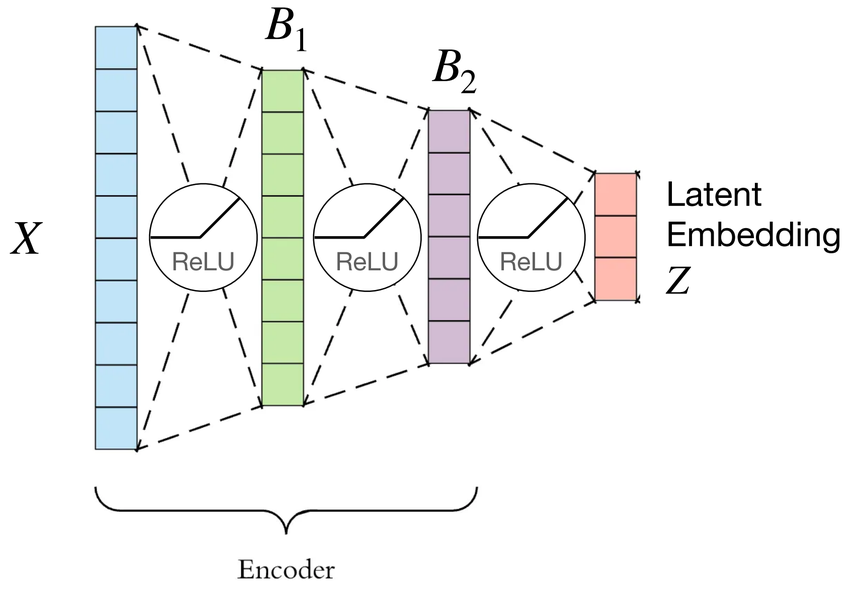
LEARNING NON-LINEAR EMBEDDINGS

💡 To capture **even more** complex semantic relationships, learning **non-linear** embeddings is required.

Neural Network Encoders

Modern encoders learn embeddings through hierarchical, structured, and compositional non-linear transformations.

- **Note:** single-layer MLP without activation is simply a linear mapping ($B = AX + \beta$).
- Each intermediate representation of an encoder is an embedding.



[7] Arden Dertat. https://medium.com/towards-data-science/applied-deep-learning-part-3-autoencoders-1c083af4d798 *(accessed January 2025)*.

LEARNING NON-LINEAR EMBEDDINGS

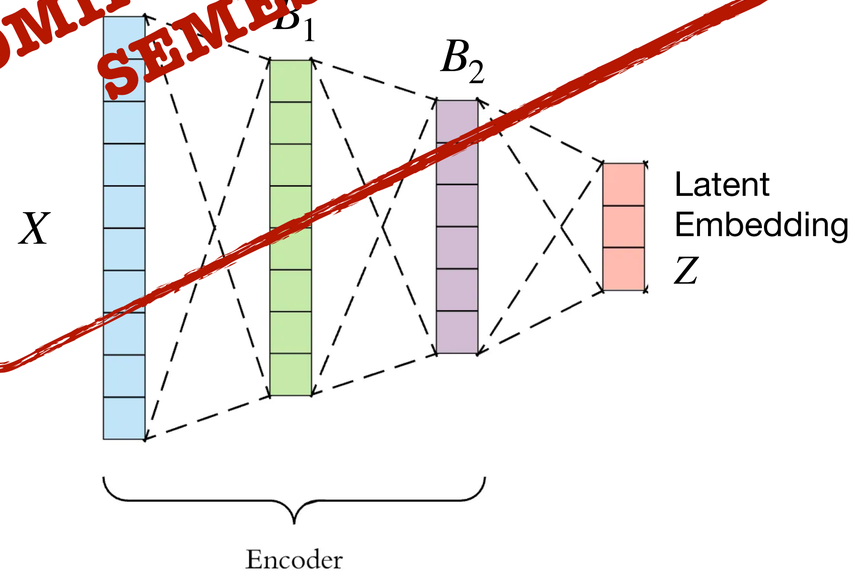
💡 To capture **even more** complex semantic relationships, learning **non-linear** embeddings is required.

Neural Network Encoders

Modern encoders learn embeddings through hierarchical, structured, and compositional non-linear transformations.

- **Note:** single-layer MLP without activation is simply a linear mapping ($B = AX + \beta$).
- Each intermediate hidden representation in an encoder is an embedding.

COMING LATER THIS SEMESTER !



[7] Arden Dertat. https://medium.com/towards-data-science/applied-deep-learning-part-3-autoencoders-1c083af4d798 *(accessed January 2025)*.

REPRESENTATIONS OF DATA

3

Structure/regularity and relationships between representations

➡️ How can we compare representations projected into an embedding space?

METRICS AND SIMILARITY

💡 Identifying the relationships between data points on a manifold is essential to capture its structure.

⚠️ Metrics and similarity functions allow for quantifying these properties in (*lower-dimensional)* spaces.

Definition:

Given a set X of points, a metric on X is a map $d : X \times X \to \mathbb{R}^+$ that is non-negative, symmetric, satisfies $d(i, i) = 0$ for all $i \in X$ and the triangle inequality holds, i.e.:

$$d(i, j) \leq d(i, k) + d(k, j) \quad \forall i, j, k \in X\,.$$

What's important in practice?

- Different functions emphasize different aspects of the data (*e.g. Euclidean distance vs. angular relationship*).
- Functions are more or less sensitive to characteristics of the data (*e.g. outliers, scale of features*).
- Measuring distances and similarities across different spaces is fundamentally distinct from doing so within the same space (*e.g. when comparing latent representations*).

[10] Harald Räcke. https://home.ttic.edu/~harry/teaching/pdf/lecture1.pdf *(accessed January 2025)*

EXAMPLES OF METRICS ON VECTOR SPACES

Manhattan Distance (L1 Norm) $$d(x, y) = \sum_{i=1}^{n} |x_i - y_i|$$

- Properties: robust to outliers
- Usage: sparse data representations, such as grid-based data

Euclidean Distance (L2 Norm) $$d(x, y) = \|x - y\|_2 = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

- Properties: sensitive to very large/small values
- Usage: common in tasks like k-nearest neighbors, clustering, and when geometric proximity matters.
- ⚠️ Minkowski Metric is the generalization of the L1 and L2 norms allowing to control sensitivity to large differences.

SIMILARITY FUNCTIONS

Mahalanobis Distance

$$d(x, y) = \sqrt{(x - y)^T \Sigma^{-1} (x - y)}$$

- Properties: takes into account dataset covariance (*important especially for non-i.i.d. data*)
- Usage: for correlated features or multi-dimensional data with different scales (*e.g. multivariate Gaussian*).

Cosine Similarity

$$\cos \operatorname{sim}(x, y) = \frac{x \cdot y}{\|x\|_2 \|y\|_2}$$

- Properties: measures angle between vectors (*not their magnitudes*), for $x \in \mathbb{R}^d : \cos \operatorname{sim}(x, y) \to 0$ as $d \to \infty$
- Usage: well-suited for tasks involving sparse vectors (*e.g. word embeddings*)

DIVERGENCES

- Metrics & Similarities compare individual representation vectors (e.g., d(point1, point2))
- Sometimes, we need to compare entire distributions of representations (e.g., "How different is the distribution of representations for real images compared to generated images?")
- Divergences provide a way to measure this difference between probability distributions.
  - Measure the "distance" or difference between two probability distributions, P and Q.
  - Key properties: Usually non-negative and positive D(P || Q) = 0 if P=Q.
  - Unlike metrics: Often not symmetric (D(P || Q) ≠ D(Q || P)) and don't satisfy the triangle inequality.
- Example: Kullback-Leibler (KL) Divergence
  - $\mathrm{KL}(P \parallel Q) = \int p(x) \log \frac{p(x)}{q(x)} \mathrm{d}x = \mathbb{E}_p \left[\log \frac{p(x)}{q(x)} \right]$, p and q denote the probability densities of P and Q
  - Expectation of the log-likelihood ratio
  - Evaluating Generative Models: Comparing the distribution of generated data (often via their representations) to real data (e.g., related to metrics like FID).
  - Shaping Representation Spaces: Used as a loss term to encourage learned representations (e.g., in Variational Autoencoders - VAEs) to follow a desired distribution (like a simple Gaussian)
  - Training models like GANs and Diffusion Models

SUMMARY SLIDE

Recap goal of the lecture

Feature extraction is transformation of inputs into a suitable “coordinate system” where the “basis vectors” are re-usable and efficient

Re-usable and efficient: Group similar inputs under common features instead of storing a separate representation for every example

Hand-crafted / linear: easy to interpret, but often lack expressive power

Non-linear / data-driven: bases are powerful and form the foundation of modern deep learning

Goal of Deep Learning:

Learn a structured feature space where geometry = semantics (king – man + woman ≈ queen).

Evaluating & Shaping Representations

Measuring the similarity between representations (Frechet Inception Distance)

Guiding models to learn compact representations (next weeks)

Guiding models to learn appropriate representations (KL divergence) (next weeks)